

SEMANA 5

Trabajo práctico 05 - Pipeline de middleware en Express

1. Información y entrega

- **Modalidad:** individual.
- **Dominio obligatorio:** reservas de salas de estudio.
- **Entrega:** URL de un repositorio de GitHub.
- **Vencimiento:** lunes siguiente a las 19:59.
- **Ejecución esperada:** `npm install` y luego `npm start` .

La entrega debe extender los conocimientos de la semana 4. Las páginas, el formulario y los recursos estáticos continúan siendo necesarios, pero la evaluación se concentra en el orden, alcance y comportamiento del middleware.

2. Consigna

Crear una aplicación web para consultar salas de estudio y reservar temporalmente un turno. La aplicación debe utilizar middleware para registrar, identificar, medir, preparar y validar solicitudes antes de ejecutar los handlers finales.

Debe integrar:

- Express, EJS y layout principal;
- Morgan como middleware de terceros;
- parsers y recursos estáticos como middleware incorporado;
- al menos dos middleware personalizados globales;
- un middleware de router;
- un middleware de validación aplicado al POST;
- un `express.Router` montado bajo `/reservas` ;
- respuesta `400` ante datos inválidos;
- página `404` al final del pipeline;
- datos iniciales y altas únicamente en memoria;
- diagrama y justificación del orden en el README.

3. Estructura obligatoria

```
tp-05-salas-middleware/  
|-- public/  
|   |-- css/  
|       |-- estilos.css  
|-- src/  
|   |-- index.js  
|-- views/  
|   |-- layouts/  
|       |-- main.ejs  
|   |-- reservas/  
|       |-- lista.ejs  
|       |-- detalle.ejs  
|       |-- nueva.ejs  
|       |-- inicio.ejs  
|       |-- no-encontrado.ejs  
|-- .gitignore  
|-- package.json  
|-- package-lock.json  
|-- README.md
```

El router permanecerá dentro de `src/index.js` . Separarlo en otro archivo será parte de la semana siguiente.

4. Configuración

Instalar:

```
npm install express ejs express-ejs-layouts morgan
```

Configurar CommonJS y scripts:

```
{  
  "type": "commonjs",  
  "scripts": {  
    "start": "node src/index.js",  
    "check": "node --check src/index.js"  
  }  
}
```

Las dependencias deben aparecer en `dependencies` . `node_modules` y `.env` deben quedar excluidos mediante `.gitignore` .

5. Datos iniciales

Definir en `src/index.js` al menos cuatro reservas iniciales propias.

Propiedad	Tipo esperado	Regla
<code>id</code>	Número	Único y creciente
<code>estudiante</code>	Texto	No vacío después de <code>trim()</code>
<code>email</code>	Texto	No vacío; validación básica
<code>sala</code>	Texto	Una opción permitida
<code>fecha</code>	Texto	Fecha recibida del formulario
<code>turno</code>	Texto	<code>Mañana</code> , <code>Tarde</code> o <code>Noche</code>
<code>personas</code>	Número	Entero entre 1 y 6

Salas sugeridas:

```
const salasPermitidas = ["Sala Norte", "Sala Sur", "Sala Multimedia"];
```

No escribir altas en un archivo. Al reiniciar deben recuperarse únicamente los datos definidos al inicio.

6. Contrato de rutas

```
GET /
GET /estado
GET /reservas
GET /reservas/nueva
GET /reservas/:id
POST /reservas
```

`GET /reservas/nueva` debe declararse antes de `GET /reservas/:id` .

Inicio

Debe explicar el propósito del sitio e incluir enlaces al listado y al formulario.

Estado

`GET /estado` debe responder JSON:

```
{
  "servicio": "activo",
  "reservas": 4,
  "solicitudId": "BIB-0001"
}
```

La cantidad y el identificador variarán durante la ejecución.

Listado

Debe mostrar todas las reservas, un estado vacío alternativo y enlaces al detalle y al formulario.

Detalle

Debe buscar por `id` , renderizar todos los datos y responder `404` con HTML cuando la reserva no exista.

Formulario

Debe incluir controles etiquetados para estudiante, email, sala, fecha, turno y cantidad de personas. Cada control necesita `id` y `name` .

Procesamiento

El POST debe utilizar un middleware de validación y un handler final diferente:

```
reservasRouter.post("/", validarReserva, crearReserva);
```

El validador responde `400` o prepara datos y llama a `next()` . El handler agrega en memoria y redirige.

7. Middleware obligatorio y orden

El pipeline global debe respetar esta dependencia mínima:

```
Morgan
-> identificarSolicitud
-> medirDuracion
-> expressLayouts
-> express.static
-> express.urlencoded
-> express.json
-> rutas de aplicación
-> router de reservas
-> página 404
```

Se puede colocar `express.static` antes de Morgan si el README explica que los recursos exitosos no quedarán registrados. Cualquier otro cambio de orden debe estar justificado por los datos que consume cada función.

8. Morgan

Registrar Morgan con un formato documentado:

```
const morgan = require("morgan");
app.use(morgan("dev"));
```

La terminal debe mostrar solicitudes con estados `200` , `302` , `400` y `404` durante la matriz de pruebas.

9. Middleware personalizado global

Identificador

Crear `identificarSolicitud` para producir valores consecutivos:

```
BIB-0001
BIB-0002
BIB-0003
```

El valor debe guardarse en `res.locals.solicitudId` , estar disponible en vistas y aparecer en `GET /estado` .

Medición

Crear `medirDuracion` para:

1. guardar el tiempo inicial;
2. registrar un listener de `finish` en `res` ;
3. continuar con `next()` ;
4. calcular milisegundos al finalizar;
5. imprimir ID, método, `req.originalUrl` , `res.statusCode` y duración.

No se acepta una duración calculada antes de que termine la respuesta.

10. Router y middleware de área

Crear y montar:

```
const reservasRouter = express.Router();
app.use("/reservas", reservasRouter);
```

Antes de las rutas internas, registrar un middleware de router que defina:

```
res.locals.seccion = "Reservas de salas";
```

La vista de listado y la del formulario deben utilizar `seccion` . La ruta `/estado` no debe depender de ese valor.

Las rutas internas deben ser relativas al montaje: `/` , `/nueva` y `/:id` . No duplicar `/reservas` dentro del router.

11. Middleware de validación

`validarReserva` debe:

1. normalizar textos con `trim()` ;
2. convertir `personas` con `Number` ;
3. comprobar campos obligatorios;
4. comprobar que la sala pertenezca a `salasPermitidas` ;
5. comprobar que el turno sea permitido;
6. comprobar un entero entre 1 y 6;
7. comprobar de manera básica que el email contenga `@` ;
8. responder `400` y conservar valores ante error;
9. mostrar un mensaje con `role="alert"` ;
10. preparar `req.reservaValidada` y ejecutar `next()` ante éxito.

La validación HTML no reemplaza estas comprobaciones del servidor.

Camino inválido

```
validarReserva  
-> status 400  
-> render del formulario  
-> fin del ciclo
```

Camino válido

```
validarReserva  
-> req.reservaValidada  
-> next()  
-> crearReserva  
-> redirect /reservas
```

El handler `crearReserva` no debe repetir toda la validación.

12. Página 404

Registrar la respuesta final después de todas las rutas y del router:

```
app.use((req, res) => {  
  res.status(404).render("no-encontrado", {  
    titulo: "Página no encontrada",  
    mensaje: "La dirección solicitada no existe.",  
  });  
});
```

No llamar a `next()` después del render.

13. Vistas y accesibilidad básica

Mantener los requisitos de la semana anterior:

- layout con idioma, codificación, viewport y título dinámico;
- encabezado, navegación, `main` y pie;
- datos mostrados con salida escapada;
- `label` asociado a cada control;
- foco visible mediante CSS;
- mensajes de error con `role="alert"` ;
- enlaces que permitan recorrer el sitio sin escribir URLs manualmente;
- identificador de solicitud visible de forma discreta en el pie.

14. Diagrama obligatorio del README

Incluir un diagrama real del POST válido. Debe usar los nombres exactos del proyecto:

```
POST /reservas
  ↓ morgan("dev")
  ↓ identificarSolicitud
  ↓ medirDuracion
  ↓ expressLayouts
  ↓ express.urlencoded
  ↓ reservasRouter
  ↓ prepararAreaReservas
  ↓ validarReserva
  ↓ crearReserva
  ↓ 302 /reservas
  ↓ finish: ID + estado + duración
```

Agregar un segundo diagrama del POST inválido e indicar en qué función termina.

15. Matriz de pruebas

Caso	Estado esperado	Evidencia
Inicio	200	Navegación y solicitud ID
Estado	200	JSON con cantidad e ID
Listado	200	Cuatro o más reservas
Estado vacío	200	Mensaje alternativo
Formulario	200	Controles etiquetados
Detalle válido	200	Datos completos
Detalle inexistente	404	Página HTML
Campos vacíos	400	Error y valores conservados
Sala no permitida	400	No se crea registro
Turno no permitido	400	No se crea registro
Email sin @	400	No se crea registro
Personas igual a 0	400	No se crea registro
Personas igual a 7	400	No se crea registro
Reserva válida	302 y luego 200	Nueva tarjeta
URL inexistente	404	Middleware final
Reinicio	200	Regreso a datos iniciales

Para cada caso revisar también Morgan y la línea de medición personalizada.

16. README

Incluir:

```
# Trabajo práctico 05

## Descripción
## Instalación
## Ejecución
## Rutas
## Pipeline de middleware
## Alcance de cada función
## Validación
## Pruebas manuales
## Persistencia temporal
```

Explicar con palabras propias:

- diferencia entre middleware incorporado, de terceros y personalizado;

- cuándo se utiliza `next()` ;
- por qué los parsers aparecen antes de la validación;
- diferencia entre alcance global, de router y de ruta;
- motivo del evento `finish` ;
- resultado del montaje del router;
- diferencia entre el POST 302 y el GET posterior;
- motivo por el cual las altas desaparecen al reiniciar.

17. Criterios de evaluación

Criterio	Puntaje
Configuración, scripts y páginas base	1 punto
Morgan y middleware incorporado	1 punto
Identificador global mediante <code>res.locals</code>	1 punto
Medición correcta hasta <code>finish</code>	1,5 puntos
Router y middleware de área	1,5 puntos
Validación de ruta y caminos excluyentes	2 puntos
Contrato HTTP y matriz de pruebas	1 punto
README, diagramas y justificación del orden	1 punto
Total	10 puntos

18. Fuera de alcance

No se evaluarán:

- routers separados en archivos;
- controladores o servicios;
- middleware centralizado de errores;
- autenticación o autorización;
- sesiones o cookies;
- bases de datos;
- escritura de reservas en archivos;
- sanitización avanzada;
- limitación de solicitudes;
- seguridad de producción;
- pruebas automatizadas.

Agregar funcionalidades fuera de alcance no reemplaza un pipeline obligatorio incompleto.

19. Comprobación final

Antes de enviar:

- ejecutar `npm run check` ;
- iniciar con `npm start` ;
- recorrer páginas y enlaces;
- probar estados 200, 302, 400 y 404;
- confirmar que ningún caso queda cargando;
- confirmar que no aparecen errores de cabeceras enviadas;
- comprobar que `req.body` existe antes de validar;
- verificar ID y medición en la terminal;
- contrastar el README con el orden real;
- reiniciar y comprobar la memoria temporal;
- verificar `package-lock.json` y ausencia de `node_modules` en Git.